

Báo cáo

Nâng cao 1. Reverse Engineering với x64dbg/IDA

Mẫu 3

MỤC LỤC

1. Phân tích tĩnh với IDA.....	3
1.1 Thông tin về mẫu.....	3
1.2 Nhận định.....	11
2. Phân tích động.....	12
3. Kết luận.....	15

1. Phân tích tĩnh với IDA

1.1 Thông tin về mẫu

a. Tên file: TaskScheduler.exe.

b. Kiểu file: PE32 executable for MS Windows 6.00 (console), Intel i386, 5 sections.

c. Các API đáng ngờ

Dựa vào mục Imports của IDA, có thể thấy chương trình đã nạp các API từ 3 thư viện kernel32.dll, user32.dll, advapi32.dll, ole32.dll, oleaut32.dll.

Các API đáng ngờ: GetStartupInfoW, IsDebuggerPresent, GetModuleFileName, CreateFileW, WriteFile, GetUserNameA.

- Nhóm COM: CoInitializeEx, CoInitializeSecurity, CoCreateInstance, CoUninitialize, SysAllocString, SysFreeString, VariantInit, VariantClear

Address	Ordinal	Name	Library
KERNEL32			
00410018		CloseHandle	KERNEL32
0041000C		CreateFileW	KERNEL32
0041004C		DecodePointer	KERNEL32
00410098		DeleteCriticalSection	KERNEL32
00410048		EncodePointer	KERNEL32
0041007C		EnterCriticalSection	KERNEL32
0041006C		ExitProcess	KERNEL32
00410028		FlushFileBuffers	KERNEL32
004100B4		FreeEnvironmentStringsW	KERNEL32
004100EC		GetACP	KERNEL32
004100F4		GetCPlInfo	KERNEL32
00410050		GetCommandLineW	KERNEL32
00410024		GetConsoleCP	KERNEL32
00410020		GetConsoleMode	KERNEL32
004100C8		GetCurrentProcess	KERNEL32
004100A8		GetCurrentProcessId	KERNEL32
00410088		GetCurrentThreadId	KERNEL32
004100B0		GetEnvironmentStringsW	KERNEL32
00410094		GetFileType	KERNEL32
00410034		GetLastError	KERNEL32
00410010		GetModuleFileNameW	KERNEL32
00410070		GetModuleHandleExW	KERNEL32
004100E0		GetModuleHandleW	KERNEL32
004100F0		GetDeviceMCP	KERNEL32
00410074		GetProcAddress	KERNEL32
0041008C		GetProcessHeap	KERNEL32
0041009C		GetStartupInfoW	KERNEL32
00410090		GetStdHandle	KERNEL32
0041002C		GetStringTypeW	KERNEL32
004100AC		GetSystemTimeAsFileTime	KERNEL32
00410060		HeapAlloc	KERNEL32
0041005C		HeapFree	KERNEL32
004100F8		HeapReAlloc	KERNEL32
00410078		HeapSize	KERNEL32
004100C0		InitializeCriticalSectionAndSpinCount	KERNEL32
00410064		IsDebuggerPresent	KERNEL32
00410068		IsProcessorFeaturePresent	KERNEL32
004100E8		IsValidCodePage	KERNEL32

Các API IsDebuggerPresent, GetStartupInfoW không được gọi từ main nên có thể bỏ qua.

xrefs to IsDebuggerPresent				
Direction	Type	Address	Text	
Up	r	raise_securityfailure+3	call	ds:IsDebuggerPresent
Up	p	__raise_securityfailure+3	call	ds:IsDebuggerPresent
Up	r	__call_reportfault+EA	call	ds:IsDebuggerPresent
Up	p	__call_reportfault+EA	call	ds:IsDebuggerPresent
Up	r	sub_40B14E:loc_40B241	call	ds:IsDebuggerPresent
Up	p	sub_40B14E:loc_40B241	call	ds:IsDebuggerPresent

xrefs to GetStartupInfoW				
Direction	Type	Address	Text	
Up	r	__init+93	call	ds:GetStartupInfoW
Up	p	__init+93	call	ds:GetStartupInfoW

Ngoài ra có LoadLibraryExW và GetProcAddress thường được dùng để nạp thêm API từ nguồn khác ([tham khảo](#)). Ở đây, Chúng được dùng để gọi MessageBoxW.


```

v2 = CoInitializeSecurity(0, -1, 0, 0, 6u, 3u, 0, 0, 0);
if ( v2 < 0 )
{
    printf("\nCoInitializeSecurity failed: %x", v2);
    CoUninitialize();
    return 1;
}
GetModuleFileNameW(0, Filename, 0x104u);
sub_401B70(Block, Filename);
v65 = 0;
ppv = 0;

```

(Hàm xử lý sub_401B70 và các hàm được gọi bên trong nó là các hàm xử lý wide string nên có thể bỏ qua logic cục bộ của sub_401B70)

```

1  DWORD * __thiscall std_wstring_init( DWORD *this, char *Src)
2  {
3      this[5] = 7;
4      this[4] = 0;
5      *(_WORD *)this = 0;
6      if ( *(_WORD *)Src )
7          sub_401ED0(this, Src, wcslen((const unsigned __int16 *)Src));
8      else
9          sub_401ED0(this, Src, 0);
10     return this;
11 }

```

Vậy sau hàm sub_401B70, path của chương trình được lưu trong biến Block.

Tiếp theo, biến v3 được gán với hàm [CoCreateInstance](#) - khởi tạo 1 đối tượng COM với CLSID từ &rclsid và giao diện &riid, giao diện đích sẽ trả về trong &ppv.

```

83  GetModuleFileNameW(0, Filename, 0x104u);
84  std_wstring_init(Block, Filename);
85  v65 = 0;
86  ppv = 0;
87  v3 = CoCreateInstance(&rclsid, 0, 1u, &riid, &ppv);
88  if ( v3 >= 0 )

```

Xem dữ liệu của các hằng số này trong disassembly.

```

.rdata:00410FE4 ; const IID rclsid
.rdata:00410FE4 rclsid IID <0F87369Fh, 0A4E5h, 4CFCh, <0BDh, 3Eh, 73h, 0E6h, 15h, 45h, 72h, \
.rdata:00410FE4 ; DATA XREF: _wmain+F8to
.rdata:00410FE4 0DDh>>
.rdata:00410FE4 ; DATA XREF: _wmain+140FA

```

Giá trị của CLSID là: {0F87369F-A4E5-4CFC-BD3E-73E6154572DD}

Trong Registry, CLSID này tương ứng với TaskScheduler class. (HKEY_CLASSES_ROOT\CLSID\)

Path	Name	Type	Data
{0F87369F-a4e5-4cfc-bd3e-73e6154572dd}	(Default)	REG_SZ	TaskScheduler class
{0F92030A-CBFD-4A88-A164-FF5985547FF6}			
{0F87369F-a4e5-4cfc-bd3e-73e6154572dd}	InprocServer32		
	ProgID		
	Programmable		
	TypeLib		
	Version		
	VersionIndependentProgID		
	(Default)	REG_SZ	C:\Windows\system32\taskschd.dll
	ThreadingModel	REG_SZ	Both

Giá trị của riid là: {2FABA4C7-4DA9-4013-9697-20CC3FD40F85}

```

.rdata:00410FD4 ; const IID riid
.rdata:00410FD4 riid IID <2FABA4C7h, 4DA9h, 4013h, <96h, 97h, 20h, 0CCh, 3Fh, 0D4h, 0Fh, \
.rdata:00410FD4 ; DATA XREF: _wmain+EFfo
.rdata:00410FD4 85h>>
.rdata:00410FE4 : const IID rclsid

```

Trong Registry, riid này tương ứng với ITaskService (HKEY_CLASSES_ROOT\Interface\).

Name	Type	Data
(Default)	REG_SZ	ITaskService

Vậy đối tượng COM được khởi tạo sử dụng giao diện `ITaskService`.

Quay lại mã giả, trong trường hợp v3 nhận khởi tạo thành công, đầu tiên, chương trình khởi tạo 1 số [Variant](#). Sau đó, các variant được xử lý (nạp vào RAM...) bởi *mm load si128* - hàm hệ thống của intel.

```
v3 = CoCreateInstance(&rclsid, 0, 1u, &riid, &ppv);
if ( v3 >= 0 )
{
    VariantInit(&pvarg);
    si128 = __mm_load_si128((const __m128i *)&pvarg);
    VariantInit(&v45);
    v47 = __mm_load_si128((const __m128i *)&v45);
    VariantInit(&v49);
    v48 = __mm_load_si128((const __m128i *)&v49);
    VariantInit(&v43);
    LOBYTE(v65) = 4;
    v41 = __mm_load_si128(&si128);
    v39 = __mm_load_si128(&v47);
    v37 = __mm_load_si128(&v48);
    v4 = __mm_loadu_si128((const __m128i *)&v43);
}
```

Tiếp theo, ppv (con trỏ tới giao diện ITaskService) được thao tác. Các đoạn mã tiếp theo là logic xử lý của ITaskService để thiết lập task.

Vì ppv là con trỏ đến [ITaskService](#), (*(DWORD *)ppv + 40) tức dịch offset 40 bytes -> trỏ đến phương thức [ITaskService::Connect](#), tạo kết nối giữa chương trình yêu cầu với dịch vụ Task Scheduler.

```
v5 = (*(int (*)[_stdcall **])(LPVOID, __int32, __int32, __int32, __int32, __int32, __int32, __int32, __int32, __int32, __int32, __int32, __int32, __int32, __int32, __int32, __int32, __int32))(*(_DWORD *)ppv + 40));  
    ppv;  
    v4.m128i_132[0],  
    v4.m128i_132[1],  
    v4.m128i_132[2],  
    v4.m128i_132[3],  
    v37.m128i_132[0],  
    v37.m128i_132[1],  
    v37.m128i_132[2],  
    v37.m128i_132[3],  
    v39.m128i_132[0],  
    v39.m128i_132[1],  
    v39.m128i_132[2],  
    v39.m128i_132[3],  
    v41.m128i_132[0],  
    v41.m128i_132[1],  
    v41.m128i_132[2],  
    v41.m128i_132[3]]);  
VariantClear(&v43);  
VariantClear(&v40);  
VariantClear(&v45);  
LOBYTE(v65) = 0;  
VariantClear(&pvarg);
```

Nếu kết nối được thiết lập thành công, thực hiện:

```

if ( v5 >= 0 )
{
    v50 = 0;
    v7 = (int **)sub_401330((OLECHAR *)"\\");
    LOBYTE(v65) = 5;
    v8 = *v7;
    if ( v8 )
        v9 = *v8;
    else
        v9 = 0;
    v10 = (*(int (__stdcall **)(LPVOID, int, void **))(*(_DWORD *)ppv + 28))(ppv, v9, &v50);
    LOBYTE(v65) = 0;
    v11 = v10;
    sub_4013E0(&v42);
    if ( v11 >= 0 )
    {
        v12 = (int **)sub_401330((OLECHAR *)L"TaskScheduler");
        LOBYTE(v65) = 6;
        v13 = *v12;
        if ( v13 )
            v14 = *v13;
        else
            v14 = 0;
        (*(void (__stdcall **)(void *, int, _DWORD))(*(_DWORD *)v50 + 60))(v50, v14, 0);
        LOBYTE(v65) = 0;
        sub_4013E0(&v42);
        v51 = 0;
        v15 = (*(int (__stdcall **)(LPVOID, _DWORD, void **))(*(_DWORD *)ppv + 36))(ppv, 0, &v51);
        (*(void (__stdcall **)(LPVOID))(*(_DWORD *)ppv + 8))(ppv);
    }
}

```

Biến v9 chứa path root ("\\").

Ở đầu vào của v10, con trỏ ppv được dùng để gọi phương thức số 7 (offset 28 bytes) - [ITaskService::GetFolder](#). Kết quả được trả về trong v50 - giao diện [ITaskFolder](#) cho folder đích.

Tiếp theo, v14 chứa chuỗi "TaskScheduler".

v50 - ITaskFolder được dịch offset 60 bytes - gọi đến phương thức số 15 - [ITaskFolder::DeleteTask](#). Mục đích của việc này là xóa các task có trùng tên.

Ở biến v15, thực hiện dịch offset ppv 36 bytes - gọi đến phương thức số 9 - [ITaskService::NewTask](#), trả về con trỏ giao diện [ITaskDefinition](#) trong v51.

Sau đó, dịch offset ppv 8 bytes - gọi đến phương thức số 2 - ITaskService::Release - giải phóng ppv.

Nếu v15 thành công, thực hiện:

```

if ( v15 >= 0 )
{
    v57 = 0;
    v16 = (*(int (__stdcall **)(void *, int **))(*(_DWORD *)v51 + 36))(v51, &v57);
    if ( v16 >= 0 )
    {
        v54 = 0;
        v18 = (*(int (__stdcall **)(int, int, int **))(*(_DWORD *)v57 + 40))(v57, 9, &v54);
        (*(void (__stdcall **)(int **))(*(_DWORD *)v57 + 8))(v57);
        if ( v18 >= 0 )
        {
            v56 = 0;

```

Nếu tạo task mới thành công, dịch v51 - con trỏ đến ITaskDefinition 36 bytes - gọi đến phương thức số 9 - ITaskDefinition::get_Triggers, trả về con trỏ các trigger (mỗi kích hoạt task)/giao diện ITriggerCollection trong v57.

Dịch v57 40 bytes - gọi đến phương thức số 10 - [ITriggerCollection::Create](#) - khởi tạo 1 mỗi kích hoạt mới, trả về con trỏ giao diện ITrigger trong v54. Sau đó giải phóng v57.

Nếu tạo mỗi kích hoạt thành công, tại v19, gọi đến phương thức đầu - [QueryInterface](#). Phương thức này truy vấn xem giao diện có tham chiếu riid (tham số tại v19 là &unk_410FF4) có được hỗ trợ bởi đối tượng hiện tại hay không, trả về con trỏ đến giao diện đó, nếu có (ở đây là v56).

```

v19 = (**(int (__stdcall **))(int, void *, int *))v54)(v54, &unk_410FF4, &v56);
(*(void (__stdcall **))(int))(*(_DWORD *)v54 + 8))(v54);
if ( v19 >= 0 )
{
    pcbBuffer = 256;
    GetUserNames(Buffer, &pcbBuffer);
    v20 = (int *)sub_401290(Buffer);
    LOBYTE(v65) = 7;
    if ( *v20 )
        v21 = **v20;
    else
        v21 = 0;
    v22 = (*(int (__stdcall **))(int, int))(*(_DWORD *)v56 + 92))(v56, v21);
    LOBYTE(v65) = 0;
    v23 = v22;
    sub_4013E0(&v42);
    (*(void (__stdcall **))(int))(*(_DWORD *)v56 + 8))(v56);
    if ( v23 < 0 )
        goto LABEL_24;
    v58 = 0;
    v24 = (*(int (__stdcall **))(void *, int *))(*(_DWORD *)v51 + 68))(v51, &v58);
    if ( v24 >= 0 )

```

Giá trị của unk_410FF4 trong stack

```

.rdata:00410FF4 unk_410FF4 db 38h ; 8 ; DATA XREF: _wmain+405to
.rdata:00410FF5 db 00Eh
.rdata:00410FF6 db 0DAh
.rdata:00410FF7 db 72h ; r
.rdata:00410FF8 db 0E4h
.rdata:00410FF9 db 0FAh
.rdata:00410FFA db 3Eh ; >
.rdata:00410FFB db 4Bh ; K
.rdata:00410FFC db 08Ah
.rdata:00410FFD db 0F4h
.rdata:00410FFE db 5Dh ; ]
.rdata:00410FFF db 0
.rdata:00411000 db 9Ah
.rdata:00411001 db 0F0h
.rdata:00411002 db 2Bh ; +
.rdata:00411003 db 1Ch
.rdata:00411004 unk_411004 db 4Bh ; M ; DATA XREF: _wmain+453to

```

là {72DADE38-FAE4-4B3E-BAF4-5D009AF02B1C}. Trong Registry, đây là GUID của ILogonTrigger. (HKEY_CLASSES_ROOT\Interface\)

{72D41A3C-199F-4E5C-8BDC-CD4F8B2CC519}	Name	Type	Data
{72DADE38-FAE4-4B3E-BAF4-5D009AF02B1C}	(Default)	REG_SZ	ILogonTrigger
{72F78AC2-A1FE-4C6E-BF0B-2CA610B2B50B}			

Sau đó giải phóng v54.

Nếu truy vấn tại v19 thành công, lấy UserName lưu vào v21. Tại v22, dịch v56 92 bytes - gọi đến [ILogonTrigger::put_UserId](#) dùng v21, rồi giải phóng v56.

Tại v24, gọi đến phương thức [ITaskDefinition::get_Actions](#) của ITaskDefinition (v51). Phương thức này thu thập các hành động sẽ được thực hiện bởi task, trả về con trỏ đến IActionCollection tại v58. Nếu thành công, thực hiện:

```

v24 = (*(int (__stdcall **)(void *, int **))(_DWORD *)v51 + 68))(v51, &v58);
if ( v24 >= 0 )
{
    v55 = 0;
    v25 = (*(int (__stdcall **)(int, _DWORD, int **))(_DWORD *)v58 + 48))(v58, 0, &v55);
    (*(void (__stdcall **)(int))(_DWORD *)v58 + 8))(v58);
    if ( v25 >= 0 )
    {
        v53 = 0;
        v26 = (*(int (__stdcall **)(int, void *, int **))v55)(v55, &unk_411004, &v53);
        (*(void (__stdcall **)(int))(_DWORD *)v55 + 8))(v55);
    }
}

```

Tại v25, gọi đến IActionCollection::Create. Action được tạo ở đây là TASK_ACTION_EXEC (giá trị 0), trả về con trỏ IAction tại v55. Sau đó giải phóng con trỏ IActionCollection (v58).

Tại v26, thực hiện QueryInterface với IAction, giá trị query là unk_411004:
{4C3D624D-FD6B-49A3-B9B7-09CB3CD3F047}

```

.rdata:00411004 unk_411004 db 4Dh ; M
.rdata:00411005 db 62h ; b
.rdata:00411006 db 3Dh ; =
.rdata:00411007 db 4Ch ; L
.rdata:00411008 db 6Bh ; k
.rdata:00411009 db 0FDh
.rdata:0041100A db 0A3h
.rdata:0041100B db 49h ; I
.rdata:0041100C db 0B9h
.rdata:0041100D db 0B7h
.rdata:0041100E db 9
.rdata:0041100F db 0C8h
.rdata:00411010 db 3Ch ; <
.rdata:00411011 db 0D3h
.rdata:00411012 db 0F0h
.rdata:00411013 db 47h ; G

```

Name	Type	Data
{4C3D624D-FD6B-49A3-B9B7-09CB3CD3F047}	REG_SZ	IExecAction

Kết quả trả về con trỏ đến IExecAction trong v53. Giải phóng IAction (v55).

Nếu truy vấn thành công, v29 nhận path của chương trình từ Block.

Tại v30, gọi đến IExecAction::put_Path với path là v29, chính thức gán path của chương trình vào giao diện thực thi của đối tượng COM này.

```

if ( v26 >= 0 )
{
    v27 = Block;
    if ( v62 >= 8 )
        v27 = (void **)Block[0];
    v28 = (int **)sub_401330((OLECHAR *)v27);
    LOBYTE(v65) = 8;
    if ( *v28 )
        v29 = **v28;
    else
        v29 = 0;
    v30 = (*(int (__stdcall **)(int, int))(_DWORD *)v53 + 44))(v53, v29);
    LOBYTE(v65) = 0;
    v31 = v30;
    sub_4013E0(&v42);
    (*(void (__stdcall **)(int))(_DWORD *)v53 + 8))(v53);
}

```

Sau đó giải phóng IExecAction (v53).

Nếu thành công tại v30, thực hiện xử lý xâu và nạp.

```

if ( v31 >= 0 )
{
    v59 = 0;
    v43.vt = 8;
    v43.lVal = (LONG)SysAllocString(&psz);
    if ( !v43.lVal )
        sub_4027F0(-2147024882);
    v48 = __mm_load_si128((const __m128i *)&v43);
    VariantInit(&v45);
    v47 = __mm_load_si128((const __m128i *)&v45);
    VariantInit(&pvarg);
    LOBYTE(v65) = 11;
    si128 = __mm_load_si128((const __m128i *)&pvarg);
    v32 = (int **)sub_401330((OLECHAR *)L"TaskScheduler");
    LOBYTE(v65) = 12;
    v33 = *v32;
    if ( v33 )
        v34 = *v33;
    else
        v34 = 0;
    v40 = __mm_load_si128(&v48);
    v38 = __mm_load_si128(&v47);
    v35 = __mm_load_si128(&si128);
}

```

Tại v36, gọi đến phương thức [TaskFolder.RegisterTaskDefinition](#) với các tham số:

- path: v34, truy ngược lại chính là L"TaskScheduler" tại v32.
- definition: v51 (ITaskDefinition).
- flag: 6 - TASK_CREATE_OR_UPDATE.
- ...
- logonType: 3 - TASK_LOGON_INTERACTIVE_TOKEN (user phải logon rồi thì task mới thực thi).
- Output: task - trả về 1 đối tượng [RegisteredTask](#) tại v59.

```
v36 = (*(int (__stdcall **)(void *, int, void *, int, __int32, __int32, __int32, __int32, __int32, __int32, __int32, int, __int32, __int32, __int32, __int32, int *)))(*(DWORD *)v50 + 68))(\n    v50,\n    v34,\n    v51,\n    6,\n    v35.m128i_132[0],\n    v35.m128i_132[1],\n    v35.m128i_132[2],\n    v35.m128i_132[3],\n    v38.m128i_132[0],\n    v38.m128i_132[1],\n    v38.m128i_132[2],\n    v38.m128i_132[3],\n    3,\n    v40.m128i_132[0],\n    v40.m128i_132[1],\n    v40.m128i_132[2],\n    v40.m128i_132[3],\n    6);
```

Cuối cùng, giải phóng các con trở đã dùng, và hủy đối tượng COM.

```

    &v59);
sub_4013E0(&v42);
VariantClear(&pvarg);
VariantClear(&v45);
LOBYTE(v65) = 0;
VariantClear(&v43);
if ( v36 >= 0 )
{
    (*(void (__stdcall **)(void *)))(_DWORD *)v50 + 8))(v50);
    (*(void (__stdcall **)(void *)))(_DWORD *)v51 + 8))(v51);
    (*(void (__stdcall **)(int)))(_DWORD *)v59 + 8))(v59);
    CoUninitialize();
    v17 = 0;
    goto LABEL_25;
}

```

Đồng thời, thiết lập các thông báo lỗi nếu thực thi tại các bước không thành công.

```

        printf("\nError saving the Task : %x", v36);
    }
    else
    {
        printf("\nCannot set path of executable: %x", v31);
    }
}
else
{
    printf("\nQueryInterface call failed for IExecAction: %x", v
}
}
else
{
    printf("\nCannot create the action: %x", v25);
}
}
else
{
    printf("\nCannot get Task collection pointer: %x", v24);
}
}
else

```

1.2 Nhận định

Chương trình sử dụng các giao diện COM của Windows Task Scheduler để tạo một task tự động chạy file thực thi của chính nó khi người dùng đăng nhập, nhằm thiết lập persistence.

2. Phân tích động

Vì chương trình đã cho thuộc kiến trúc 32-bit, sử dụng x32dbg.

Đặt breakpoint tại các API đã xuất hiện trong mã giả (bỏ qua các API báo lỗi): GetModuleFileNameW, CoCreateInstance, GetUserNameA.

Type	Address	Module/Label/Exception	State	Disassembly
Software				
	7648AE6C	<advapi32.dll.GetUserNameA>	Enabled	mov edi,edi
	76614F80	<kernel32.dll.GetModuleFileNameW>	Enabled	mov edi,edi
	76BC99E7	<ole32.dll.CoCreateInstance>	Enabled	mov edi,edi

Đầu tiên, với API GetModuleFileNameW.

EIP	76614F80	8BFF	mov edi,edi	GetModuleFileNameW
	76614F82	55	push ebp	
	76614F83	8BEC	mov ebp,esp	
	76614F85	5D	pop ebp	
	76614F86	EB ED	jmp <JMP.&GetModuleFileNameW>	

Các tham số trong stack: Tham số thứ 1 và thứ 3 là các hằng số, có giá trị lần lượt là 0 và 104.

Default (stdcall)	
1:	[esp+4] 00000000
2:	[esp+8] 00E38870 L"C:\\Users\\trang\\Downloads\\3.Persistence\\TaskScheduler.exe"
3:	[esp+C] 00000104
4:	[esp+10] 00000000
5:	[esp+14] 00000000

0032FB44	00E26D79	return to task scheduler.00E26D79 from ???
0032FB48	00000000	
0032FB4C	00E38870	L"C:\\Users\\trang\\Downloads\\3.Persistence\\TaskScheduler.exe"
0032FB50	00000104	
0032FB54	00000000	

Sau khi thực thi, dữ liệu lưu trong tham số thứ 2 (Filename) chính là path của chương trình hiện tại.

00E38870	43 00 3A 00	5C 00 55 00	73 00 65 00	72 00 73 00	C:.\\U.s.e.r.s.
00E38880	5C 00 74 00	72 00 61 00	6E 00 67 00	5C 00 44 00	\\.t.r.a.n.g.\\.D.
00E38890	6F 00 77 00	6E 00 6C 00	6F 00 61 00	64 00 73 00	o.w.n.l.o.a.d.s.
00E388A0	5C 00 33 00	2E 00 50 00	65 00 72 00	73 00 69 00	\\.3...P.e.r.s.i.
00E388B0	73 00 74 00	65 00 6E 00	63 00 65 00	5C 00 54 00	s.t.e.n.c.e.\\.T.
00E388C0	61 00 73 00	6B 00 53 00	63 00 68 00	65 00 64 00	a.s.k.S.c.h.e.d.
00E388D0	75 00 6C 00	65 00 72 00	2E 00 65 00	78 00 65 00	u.l.e.r...e.x.e.
00E388E0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	

Tiếp theo, với API CoCreateInstance:

EIP EAX	76BC99E7	8BFF	mov edi,edi	CoCreateInstance
	76BC99E9	55	push ebp	
	76BC99EA	8BEC	mov ebp,esp	
	76BC99EC	83EC 0C	sub esp,C	
	76BC99EF	56	push esi	

Các tham số trong stack:

Default (stdcall)		
1:	[esp+4]	00360FE4 taskscheduler.00360FE4
2:	[esp+8]	00000000
3:	[esp+C]	00000001
4:	[esp+10]	00360FD4 taskscheduler.00360FD4
5:	[esp+14]	002EF954

002EF8A4	0035155E	return to taskscheduler.0035155E from ???
002EF8A8	00360FE4	taskscheduler.00360FE4
002EF8AC	00000000	
002EF8B0	00000001	
002EF8B4	00360FD4	taskscheduler.00360FD4
002EF8B8	002EF954	
002EF8BC	81207300	
002EF8C0	00000000	
002EF8C4	00000000	
002EF8C8	00538410	
002EF8CC	01010301	
002EF8D0	002EF8B0	

Cho chương trình Execute till return của API này.

Tham số thứ nhất - rlclsid có giá trị trong dump là (hex) 9F 36 87 0F E5 A4 FC 4C BD 3E 73 E6 15 45 72 DD, trùng khớp với CLSID khi phân tích mã giả:

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Watch 1	[x=] Locals
Address	Hex				ASCII	
00360FE4	9F 36 87 0F	E5 A4 FC 4C	BD 3E 73 E6	15 45 72 DD	.6..ã=ÜL%>æ.ErÝ	
00360FF4	38 DE DA 72	E4 FA 3E 4B	BA F4 5D 00	9A F0 2B 1C	8pÚrãú>K°ô]..ð+.	
00361004	4D 62 3D 4C	6B FD A3 49	B9 87 09 CB	3C D3 F0 47	Mb=Lký£I'..É<ôðG	
00361014	6C 5A 36 00	A0 31 35 00	63 73 6D E0	01 00 00 00	1Z6. 15.csmã....	
00361024	00 00 00 00	00 00 00 00	03 00 00 00	20 05 93 19	?5. Z6.	
00361034	00 00 00 00	00 00 00 00	00 3F 35 00	B4 5A 36 00		

Tương tự, tham số thứ 4 - riid có giá trị trong dump là C7 A4 AB 2F A9 4D 13 40 96 97 20 CC 3F D4 0F 85, trùng khớp với riid trong mã giả.

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Watch 1	[x=] Locals
Address	Hex				ASCII	
00360FD4	C7 A4 AB 2F	A9 4D 13 40	96 97 20 CC	3F D4 0F 85	ÇP«/EM.@.. I?ò..	
00360FE4	9F 36 87 0F	E5 A4 FC 4C	BD 3E 73 E6	15 45 72 DD	.6..ã=ÜL%>æ.ErÝ	
00360FF4	38 DE DA 72	E4 FA 3E 4B	BA F4 5D 00	9A F0 2B 1C	8pÚrãú>K°ô]..ð+.	
00361004	4D 62 3D 4C	6B FD A3 49	B9 87 09 CB	3C D3 F0 47	Mb=Lký£I'..É<ôðG	
00361014	6C 5A 36 00	A0 31 35 00	63 73 6D E0	01 00 00 00	1Z6. 15.csmã....	
00361024	00 00 00 00	00 00 00 00	03 00 00 00	20 05 93 19	?5. Z6.	

Cuối cùng, với API GetUserNameA.

EIP	7648AE6C	8BFF	mov edi,edi	GetUserNameA
	7648AE6E	55	push ebp	
	7648AE6F	8BEC	mov ebp,esp	
	7648AE71	FF75 0C	push dword ptr ss:[ebp+C]	[ebp+C]:&L"C:\\Users\\

Với hàm này, cần theo dõi tham số thứ nhất (lpBuffer) - địa chỉ trong stack là 0045F7A0:

Default (stdcall)		
1:	[esp+4]	0045F7A0 "PQRSb"
2:	[esp+8]	0045F784
3:	[esp+C]	F1F287CC
4:	[esp+10]	00000000
5:	[esp+14]	00000000

0045F6C0	0001189E	return to taskscheduler.0001189E from ???
0045F6C4	0045F7A0	"PQRSb"
0045F6C8	0045F784	
0045F6CC	F1F287CC	
0045F6D0	00000000	
0045F6D4	00000000	

Trước khi thực thi hàm, giá trị trong dump của tham số này là 1 giá trị rác (PQRSb)

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Water
Address	Hex				ASCII
0045F7A0	50 51 52 53	62 00 2C 40	58 59 5A 7B	7C 7D 7E 7F	PQRSb.,
0045F7B0	00 00 00 00	84 85 86 87	00 00 00 00	8C 8D 8E 00	
0045F7C0	00 00 52 00	94 95 96 97	10 F7 45 00	8C 9D 8E 9F	..R....

Sau khi Execute till return, giá trị lúc này chính là username hiện tại (trang)

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Water
Address	Hex				ASCII
0045F7A0	74 72 61 6E	67 00 2C 40	58 59 5A 7B	7C 7D 7E 7F	trang.,
0045F7B0	00 00 00 00	84 85 86 87	00 00 00 00	8C 8D 8E 00	
0045F7C0	00 00 52 00	94 95 96 97	10 F7 45 00	8C 9D 8E 9F	..R....

3. Kết luận

Tóm tắt hành vi:

1. Sử dụng API GetModuleFileName để lấy đường dẫn tệp thực thi của chính nó và dùng GetUserNameA để lấy tên của người dùng đang đăng nhập.
2. Khởi tạo một đối tượng COM với giao diện ITaskService để kết nối vào dịch vụ lập lịch của Windows.
3. Định nghĩa hành động thực thi (IExecAction) trỏ tới chính đường dẫn tệp của nó đã lấy ở bước đầu. Đồng thời, thiết lập môi kích hoạt (ILogonTrigger) gắn với tài khoản người dùng vừa thu thập được.
4. Tác vụ được đăng ký vào hệ thống với tên "TaskScheduler".

Kết luận: Mẫu TaskScheduler.exe thực hiện hành vi thiết lập persistence trên hệ thống thông qua việc lạm dụng Windows Task Scheduler bằng COM (Component Object Model).